

PROGRAM 1

Explore pre-trained word vectors. Explore word relationships using vector arithmetic. Perform arithmetic operations and analyze results.

PROGRAM CODE

Module or library install command (run this in terminal before running the script)

pip install gensim scipy

Import required libraries

import gensim.downloader as api # For downloading pre-trained word vectors

from scipy.spatial.distance import cosine # For calculating cosine similarity

Load pre-trained Word2Vec model (Google News, 300 dimensions)

print("Loading Word2Vec model...")

model = api.load("word2vec-google-news-300")

print("Model loaded successfully.\n")

Get and print the first 10 dimensions of the word vector for 'king'

vector = model['king']

print("First 10 dimensions of 'king' vector:")

print(vector[:10], "\n")

Print top 10 most similar words to 'king'

print("Top 10 words most similar to 'king':")

for word, similarity in model.most_similar('king'):

print(f"{word}: {similarity:.4f}")

print()

Perform word analogy: king - man + woman $\approx$ queen

result = model.most_similar(positive=['king', 'woman'], negative=['man'], topn=1)

print("Analogy - 'king' - 'man' + 'woman' $\approx$?")

print(f"Result: {result[0][0]} (Similarity: {result[0][1]:.4f})\n")

Analogy: paris + italy - france $\approx$ rome

```
print("Analogy - 'paris' + 'italy' - 'france'  $\approx$  ?")
```

```
for word, similarity in model.most_similar(positive=['paris', 'italy'],  
negative=['france']):
```

```
    print(f"{word}: {similarity:.4f}")
```

```
print()
```

Analogy: walking + swimming - walk $\approx$ swim

```
print("Analogy - 'walking' + 'swimming' - 'walk'  $\approx$  ?")
```

```
for word, similarity in model.most_similar(positive=['walking', 'swimming'],  
negative=['walk']):
```

```
    print(f"{word}: {similarity:.4f}")
```

```
print()
```

Calculate cosine similarity between 'king' and 'queen'

```
similarity = 1 - cosine(model['king'], model['queen'])
```

```
print(f"Cosine similarity between 'king' and 'queen': {similarity:.4f}")
```

OUTPUT

Loading Word2Vec model...

[=====] 100.0%

1662.8/1662.8MB downloaded

Model loaded successfully.

First 10 dimensions of 'king' vector:

```
[ 0.12597656  0.02978516  0.00860596  0.13964844 -0.02563477 -0.03613281  
 0.11181641 -0.19824219  0.05126953  0.36328125]
```

Top 10 words most similar to 'king':

kings: 0.7138

queen: 0.6511

monarch: 0.6413

crown_prince: 0.6204

prince: 0.6160

sultan: 0.5865

ruler: 0.5798

princes: 0.5647

Prince_Paras: 0.5433

throne: 0.5422

Analogy - 'king' - 'man' + 'woman' $\approx$?

Result: queen (Similarity: 0.7118)

Analogy - 'paris' + 'italy' - 'france' $\approx$?

lohan: 0.5070

madrid: 0.4818

heidi: 0.4800

real_madrid: 0.4753

florence: 0.4682

diego: 0.4673

ronnie: 0.4672

juventus: 0.4672

joel: 0.4654

huntelaar: 0.4636

Analogy - 'walking' + 'swimming' - 'walk' $\approx$?

Swimming: 0.6006

swim: 0.5949

swimmers: 0.5835

swimmer: 0.5819

paddling: 0.5744

kayaking: 0.5662

swam: 0.5506

rowing: 0.5436

swims: 0.5371

canoeing: 0.5140

Cosine similarity between 'king' and 'queen': 0.6511

PROGRAM 2

Use dimensionality reduction (e.g., PCA or t-SNE) to visualize word embeddings for Q 1. Select 10 words from a specific domain (e.g., sports, technology) and visualize their embeddings. Analyze clusters and relationships. Generate contextually rich outputs using embeddings. Write a program to generate 5 semantically similar words for a given input.

PROGRAM CODE

Module or library install command (run this in terminal before running the script)

pip install gensim matplotlib scikit-learn
import gensim.downloader as api

```
from sklearn.decomposition import PCA
```

```
import matplotlib.pyplot as plt# Load model
```

```
model = api.load("word2vec-google-news-300")# Select 10 domain-specific words  
(technology domain)
```

```
words = ['computer', 'internet', 'software', 'hardware', 'keyboard', 'mouse', 'server',  
'network', 'programming', 'database']
```

```
vectors = [model[word] for word in words]# Dimensionality reduction using PCA
```

```
pca = PCA(n_components=2)
```

```
reduced = pca.fit_transform(vectors)# Generate 5 semantically similar words for a  
given input
```

```
input_word = 'computer'
```

```
similar_words = model.most_similar(input_word, topn=5)# Print the similar words
to terminal

print(f"Top 5 words similar to '{input_word}':")

for word, score in similar_words:

print(f"{word}: {score:.4f}")# Plot the word embeddings

plt.figure(figsize=(8, 6))

for i, word in enumerate(words):

plt.scatter(reduced[i, 0], reduced[i, 1])

plt.annotate(word, (reduced[i, 0], reduced[i, 1]))

plt.title("PCA Visualization of Technology Word Embeddings")

plt.xlabel("PC1")

plt.ylabel("PC2")

plt.grid(True)# Show the plot

plt.show()
```

OUTPUT

Top 5 words similar to 'computer':

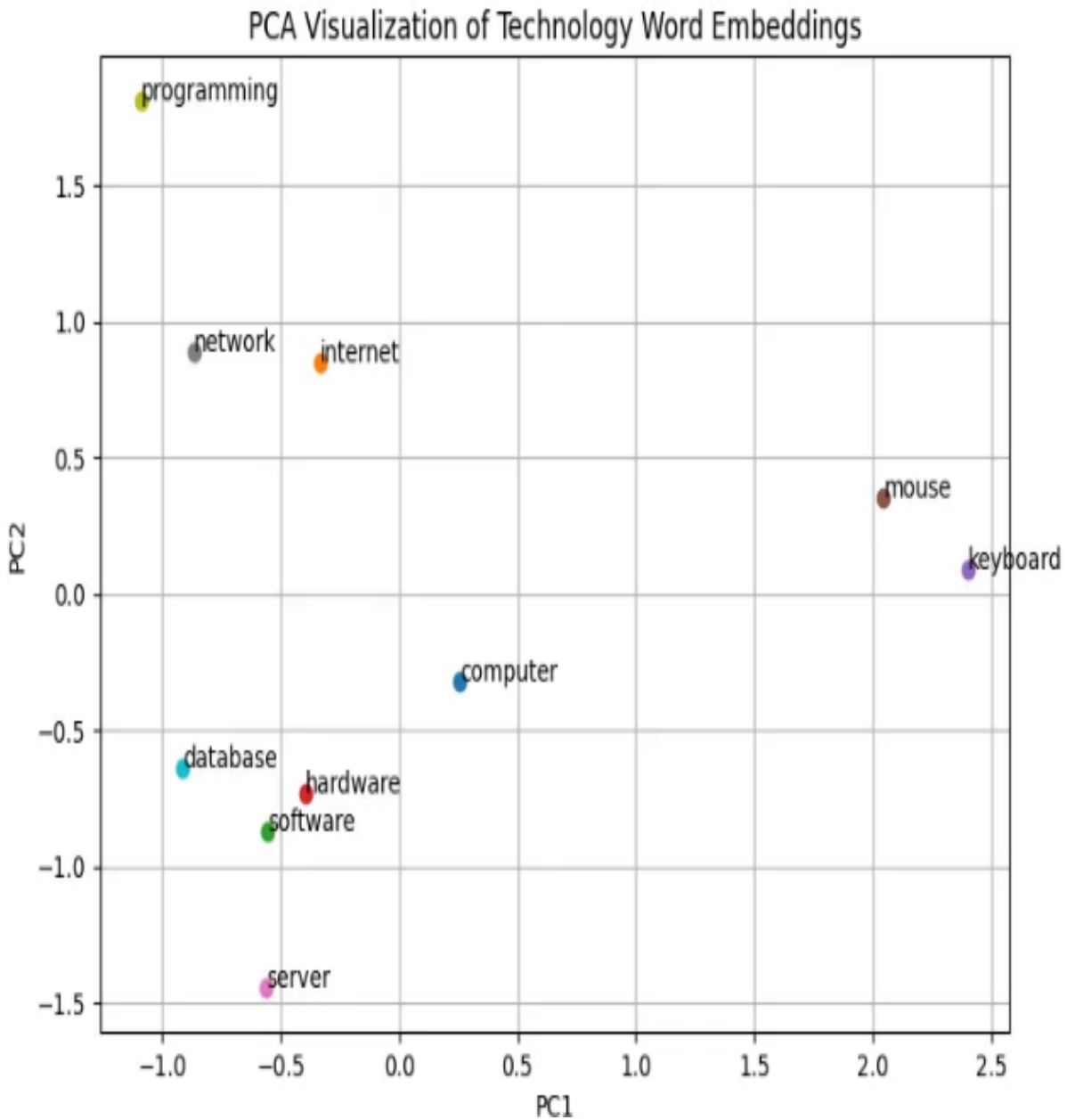
computers: 0.7979

laptop: 0.6640

laptop_computer: 0.6549

Computer: 0.6473

com_puter: 0.6082



PROGRAM 3

Train a custom Word2Vec model on a small dataset. Train embeddings on a domain-specific corpus (e.g., legal, medical) and analyze how embeddings capture domain-specific semantics.

PROGRAM CODE

Module or library install command (run this in terminal before running the script)

pip install gensim matplotlib scikit-learn

```
import matplotlib.pyplot as plt
```

```
from sklearn.manifold import TSNE
```

```
import numpy as np
```

```
from gensim.models import Word2Vec
```

Sample domain-specific corpus (medical domain)

```
medical_corpus = [
```

```
    "The patient was diagnosed with diabetes and hypertension.",
```

```
    "MRI scans reveal abnormalities in the brain tissue.",
```

```
    "The treatment involves antibiotics and regular monitoring.",
```

```
    "Symptoms include fever, fatigue, and muscle pain.",
```

```
    "The vaccine is effective against several viral infections.",
```

```
    "Doctors recommend physical therapy for recovery.",
```

```
    "The clinical trial results were published in the journal.",
```

```
    "The surgeon performed a minimally invasive procedure.",
```

```
    "The prescription includes pain relievers and anti-inflammatory drugs.",
```

```
    "The diagnosis confirmed a rare genetic disorder."
```

```
]
```

Preprocess corpus (tokenize sentences and convert to lowercase)

```
processed_corpus = [sentence.lower().split() for sentence in medical_corpus]
```

Train a Word2Vec model

```
model = Word2Vec(sentences=processed_corpus, vector_size=100, window=5,  
min_count=1, workers=4, epochs=50)
```

Extract embeddings for visualization

```
words = list(model.wv.index_to_key) # List of words in the vocabulary
embeddings = np.array([model.wv[word] for word in words]) # Word embeddings
for each word
```

Dimensionality reduction using t-SNE

```
tsne = TSNE(n_components=2, random_state=42, perplexity=5)
tsne_result = tsne.fit_transform(embeddings)
```

Visualization of word embeddings

```
plt.figure(figsize=(10, 8))
plt.scatter(tsne_result[:, 0], tsne_result[:, 1], color="blue")
```

Annotating each point with the corresponding word

```
for i, word in enumerate(words):
    plt.text(tsne_result[i, 0] + 0.02, tsne_result[i, 1] + 0.02, word, fontsize=12)
```

```
plt.title("Word Embeddings Visualization (Medical Domain)")
plt.xlabel("Dimension 1")
plt.ylabel("Dimension 2")
plt.grid(True)
```

```
plt.show()
```

Analyze domain-specific semantics

```
def find_similar_words(input_word, top_n=5):
    try:
        similar_words = model.wv.most_similar(input_word, topn=top_n)
        print(f"Words similar to '{input_word}':")
        for word, similarity in similar_words:
            print(f" {word} ({similarity:.2f})")
```


except KeyError:

```
print(f'''{input_word}' not found in vocabulary.'')
```

Generate semantically similar words

```
find_similar_words("treatment")
```

```
find_similar_words("vaccine")
```

OUTPUT

Words similar to 'treatment':

procedure. (0.27)

confirmed (0.15)

muscle (0.13)

monitoring. (0.12)

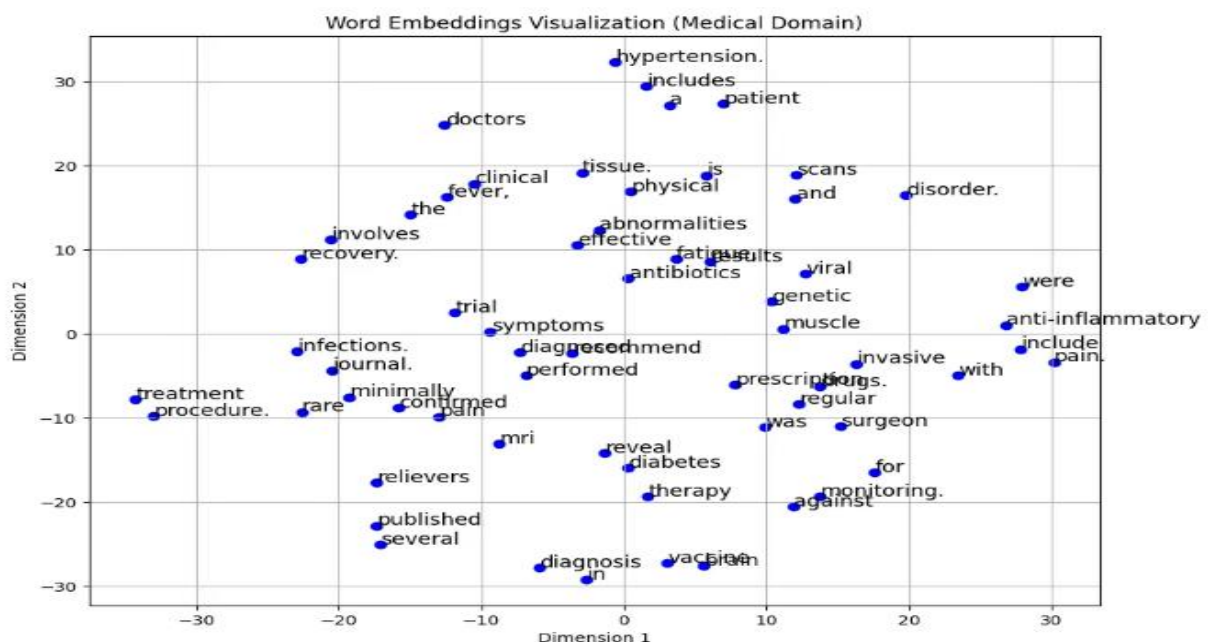
fatigue, (0.12)

Words similar to 'vaccine':

brain (0.26)

recommend (0.21)

procedure. (0.19) therapy (0.19) in (0.18)



PROGRAM 4

Use word embeddings to improve prompts for Generative AI model. Retrieve similar words using word embeddings. Use the similar words to enrich a GenAI prompt. Use the AI model to generate responses for the original and enriched prompts. Compare the outputs in terms of detail and relevance

PROGRAM CODE

Install required libraries

Install gensim for downloading pre-trained models

```
!pip install gensim
```

```
!pip install torch torchvision torchaudio
```

Install Hugging Face Transformers for NLP pipelines

```
!pip install transformers
```

Install NLTK for text preprocessing and tokenization

```
!pip install nltk
```

```
import gensim.downloader as api
```

```
from transformers import pipeline
```

```
import nltk
```

```
import string
```

```
from nltk.tokenize import word_tokenize
```

```
nltk.download('punkt')
```

Load pre-trained word vectors

```
print("Loading pre-trained word vectors...")
```

```
word_vectors = api.load("glove-wiki-gigaword-100") # 100D GloVe word embeddings
```

```
print("Word vectors loaded successfully!")
```

Function to replace words in the prompt with their most similar words

```
def replace_keyword_in_prompt(prompt, keyword, word_vectors, topn=1):
```

```

words = word_tokenize(prompt) # Tokenize the prompt into words
enriched_words = []
for word in words:
    cleaned_word = word.lower().strip(string.punctuation) # Normalize
    word
    if cleaned_word == keyword.lower(): # Replace only if it matches the
    keyword
        try:
            similar_words = word_vectors.most_similar(cleaned_word,
topn=topn)

            if similar_words:
                replacement_word = similar_words[0][0]
                print(f"Replacing '{word}' -> '{replacement_word}'")
                enriched_words.append(replacement_word)
                continue # Skip appending the original word
            except KeyError:
                print(f"'{keyword}' not found in the vocabulary. Using
original word.")
                enriched_words.append(word)
enriched_prompt = " ".join(enriched_words)
print(f"\nEnriched Prompt: {enriched_prompt}")
return enriched_prompt
print("\nLoading GPT-2 model...")
generator = pipeline("text-generation", model="gpt2")
def generate_response(prompt, max_length=100):
    try:
        response = generator(prompt, max_length=max_length,
num_return_sequences=1, truncation=True, pad_token_id=50256)

```

```
        return response[0]['generated_text']
    except Exception as e:
        print(f"Error generating response: {e}")
        return None

original_prompt = "Who is king."
print(f"\nOriginal Prompt: {original_prompt}")

key_term = "king"

enriched_prompt = replace_keyword_in_prompt(original_prompt, key_term,
word_vectors)

print("\nGenerating response for the original prompt...")
original_response = generate_response(original_prompt)
print("\nOriginal Prompt Response:")
print(original_response)

print("\nGenerating response for the enriched prompt...")
enriched_response = generate_response(enriched_prompt)
print("\nEnriched Prompt Response:")
print(enriched_response)

print("\nComparison of Responses:")
print("Original Prompt Response Length:", len(original_response))
print("Enriched Prompt Response Length:", len(enriched_response))
print("Original Prompt Response Detail:", original_response.count("."))
print("Enriched Prompt Response Detail:", enriched_response.count("."))
```

OUTPUT

Loading GPT-2 model...

Device set to use cpu

Original Prompt: Who is king.

Replacing 'king' -> 'prince'

Enriched Prompt: Who is prince .

Generating response for the original prompt...

Original Prompt Response:

Who is king. A prince shall rule over all. Who is king of nations. Who is king of countries. A prince shall rule over all. No king shall rule over any nation because he has created for himself a king.

What does this mean? For him who is king, there cannot have been a world, either a world of nations, or a world either of kings. It meant for king Solomon to reign in a world, because the kings of the world had ruled. That

Generating response for the enriched prompt...

Enriched Prompt Response:

Who is prince ... who is prince... and one of the great men is Prince Harry... and someone else is Princess Diana... that's who." (The last two sentences have been translated into Chinese by a team of translators.)

The two have come together to discuss the best and the worst of English, which is how they understand their English. And even when a translation error occurs, it's often the best thing that can happen. It's all

Comparison of Responses:

Original Prompt Response Length: 424

Enriched Prompt Response Length: 426

Original Prompt Response Detail: 8

Enriched Prompt Response Detail: 16

PROGRAM 5

Use word embeddings to create meaningful sentences for creative tasks. Retrieve similar words for a seed word. Create a sentence or story using these words as a starting point. Write a program that: Takes a seed word. Generates similar words. Constructs a short paragraph using these words.

PROGRAM CODE

Required Libraries:

Install gensim if not already installed:

pip install gensim

```
import gensim.downloader as api # For loading pre-trained word embeddings
from gensim.models import KeyedVectors # For working with word vectors
import random # For shuffling similar words
model = api.load("glove-wiki-gigaword-100")
def generate_similar_words(seed_word, topn=10):
    if seed_word in model:
        return [word for word, _ in model.most_similar(seed_word, topn=topn)]
    else:
        return []
def create_paragraph(seed_word):
    similar_words = generate_similar_words(seed_word, topn=10)
    if not similar_words:
        return f"No similar words found for '{seed_word}'."

    random.shuffle(similar_words)
    selected_words = similar_words[:5]

    paragraph = f"In a world defined by {seed_word}, "
    paragraph += f"people found themselves surrounded by concepts like {'',
'.join(selected_words[:-1])}, and {selected_words[-1]}. "
    paragraph += f"These ideas shaped the way they thought, acted, and dreamed.
Every step forward in their journey reflected the essence of '{seed_word}', "
```

```
    paragraph += f"bringing them closer to understanding the true meaning of  
{selected_words[0]}."  
  
    return paragraph  
  
seed = "freedom" # You can change this to any word like 'love', 'innovation', etc.  
print(create_paragraph(seed))
```

OUTPUT

In a world defined by freedom, people found themselves surrounded by concepts like equality, democracy, rights, free, and independence. These ideas shaped the way they thought, acted, and dreamed. Every step forward in their journey reflected the essence of 'freedom', bringing them closer to understanding the true meaning of equality.

PROGRAM 6

Use a pre-trained Hugging Face model to analyze sentiment in text. Assume a real-world application, Load the sentiment analysis pipeline. Analyze the sentiment by giving sentences to input.

PROGRAM CODE

```
# Step 1: Install required libraries (only run once)  
  
# pip install transformers torch  
  
# Step 2: Import necessary library  
from transformers import pipeline  
  
sentiment_pipeline = pipeline("sentiment-analysis")  
  
input_sentences = [  
    "The new phone I bought is absolutely amazing!",  
    "Worst customer service ever. I'm never coming back.",  
    "The experience was average, nothing special.",  
    "Fast delivery and the packaging was perfect.",  
    "The product broke within two days. Very disappointed."  
]
```

```
results = sentiment_pipeline(input_sentences)
print("Sentiment Analysis Results:\n")
for sentence, result in zip(input_sentences, results):
    print(f"Input Sentence: {sentence}")
    print(f"Predicted Sentiment: {result['label']}, Confidence Score: {result['score']:.2f}\n")
```

OUTPUT

Sentiment Analysis Results:

Input Sentence: The new phone I bought is absolutely amazing!

Predicted Sentiment: POSITIVE, Confidence Score: 1.00

Input Sentence: Worst customer service ever. I'm never coming back.

Predicted Sentiment: NEGATIVE, Confidence Score: 1.00

Input Sentence: The experience was average, nothing special.

Predicted Sentiment: NEGATIVE, Confidence Score: 1.00

Input Sentence: Fast delivery and the packaging was perfect.

Predicted Sentiment: POSITIVE, Confidence Score: 1.00

Input Sentence: The product broke within two days. Very disappointed.

Predicted Sentiment: NEGATIVE, Confidence Score: 1.00

PROGRAM 7

Summarize long texts using a pre-trained summarization model using Hugging face model. Load the summarization pipeline. Take a passage as input and obtain the summarized text.

PROGRAM CODE

Required libraries (install before running this script):

pip install transformers torch

```
from transformers import pipeline
```

```
text = """
```

The Industrial Revolution, which took place from the 18th to the 19th centuries, was a period during which predominantly agrarian, rural societies in Europe and America became industrial and urban. Prior to the Industrial Revolution, manufacturing was often done in people's homes, using hand tools or basic machines. Industrialization marked a shift to powered, special-purpose machinery, factories and mass production. The iron and textile industries, along with the development of the steam engine, played central roles in the Industrial Revolution, which also saw improved systems of transportation, communication and banking. While industrialization brought about an increased volume and variety of manufactured goods and an improved standard of living for some, it also resulted in often grim employment and living conditions for the poor and working classes.

```
"""
```

```
summary = summarizer(text, max_length=60, min_length=30, do_sample=False)
```

```
print(summary[0]['summary_text'])
```

OUTPUT

the Industrial Revolution took place from the 18th to the 19th centuries . the industrial revolution marked a shift to powered, special-purpose machinery, factories and mass production . iron and textile industries, along with the development of the steam engine, played central roles .

PROGRAM 8

Install langchain, cohere (for key), langchain-community. Get the api key(By logging into Cohere and obtaining the cohere key). Load a text document from your google drive . Create a prompt template to display the output in a particular manner.

PROGRAM CODE

Install required libraries (Run this only once)

```
!pip install langchain cohere langchain-community
```

```

import cohere
import getpass
from langchain import PromptTemplate
from langchain.llms import Cohere

# Load the Text File saved in the CWD
file_path = "C:\\Users\\Admin\\Desktop\\GENERATIVE\\Teaching.txt"
#file_path = "Teaching.txt" # absolute pathname

try:
    with open(file_path, "r", encoding="utf-8") as file:
        text_content = file.read()
        print("File loaded successfully!")
except Exception as e:
    print("Error loading file:", str(e))

# Set Up Cohere API Key by creating an account on the Cohere website
COHERE_API_KEY = getpass.getpass("Enter your Cohere API Key:")

# Initialize Cohere Model with LangChain
cohere_llm = Cohere(cohere_api_key=COHERE_API_KEY, model="command")

# Create a Prompt Template
template = """
You are an AI assistant helping to summarize and analyze a text document.
Here is the document content:
{text}
Summary:
- Provide a concise summary of the document.
"""

# Format the Prompt and Generate Output

```

```
prompt_template = PromptTemplate(input_variables=["text"], template=template)
formatted_prompt = prompt_template.format(text=text_content)
response = cohere_llm.predict(formatted_prompt)
```

Display the Generated Output

```
print("\n **Formatted Output** ")
print(response)
```

OUTPUT

File loaded successfully!

Enter your Cohere API Key:

****Formatted Output****

Engineering is a discipline that applies science, mathematics, and technology to solve practical problems and improve the quality of life. It plays a crucial role in modern society by contributing to various fields like civil, mechanical, electrical, and computer engineering. Advancements in engineering have revolutionized technology, from the internet and AI to space exploration and renewable energy. Ethical, sustainable, and safe practices ensure that engineering innovations benefit society and the environment. It continues to drive technological progress through robotics, nanotechnology, and quantum computing.

PROGRAM 9

Take the Institution name as input. Use Pydantic to define the schema for the desired output and create a custom output parser. Invoke the Chain and Fetch Results. Extract the below Institution related details from Wikipedia: The founder of the Institution. When it was founded. The current branches in the institution. How many employees are working in it? A brief 4-line summary of the institution.

PROGRAM CODE

```
from pydantic import BaseModel
from typing import List, Optional
import wikipediaapi
import requests
from bs4 import BeautifulSoup
```

```

import re

class InstitutionProfile(BaseModel):
    founder: Optional[str] = "Unknown"
    Established: Optional[int] = 0
    branches: List[str] = ["Unknown"]
    employee_count: Optional[int] = 0
    summary: str

def fetch_wikipedia_data(institution_name: str) -> dict:
    user_agent = "MyInstitutionInfoBot/1.0 (contact: your-  
email@example.com)"
    wiki_wiki = wikipediaapi.Wikipedia(user_agent=user_agent,  
language="en")
    page = wiki_wiki.page(institution_name)
    if not page.exists():
        raise ValueError(f"Wikipedia page for '{institution_name}' not  
found.")
    summary = page.summary[:300] # First 300 characters as a brief summary
    # Fetch page HTML for infobox parsing
    wiki_url = f"https://en.wikipedia.org/wiki/{institution_name.replace(' ', '_')}}"
    headers = {"User-Agent": user_agent}
    response = requests.get(wiki_url, headers=headers)
    soup = BeautifulSoup(response.text, "html.parser")
    infobox = soup.find("table", {"class": "infobox"})
    data = {
        "founder": "Unknown",
        "Established": 0,
        "branches": ["Unknown"],

```

```

        "employee_count": 0,
        "summary": summary
    }

if infobox:
    for row in infobox.find_all("tr"):
        header = row.find("th")
        value = row.find("td")
        if header and value:
            key = header.text.strip()
            val = value.text.strip()
            if key in ["Founder", "Founders", "Founder(s)"]:
                data["founder"] = val
            elif key in ["Established", "Founded", "Formation"]:
                match = re.search(r"\b(18\d{2}|19\d{2}|20\d{2})\b", val)
                if match:
                    data["Established"] = int(match.group(0))
            elif key in ["Total staff", "Employees", "Staff"]:
                match = re.search(r"(\d{3,5})", val)
                if match:
                    data["employee_count"] = int(match.group(0))
            elif key in ["Address", "Location"]:
                data["branches"] = [val.split(",")[0]] # Take first Location

# Try DBpedia as a backup if founder is missing
if data["founder"] == "Unknown":
    dbpedia_data = fetch_dbpedia_data(institution_name)
    data.update(dbpedia_data)

```

```

        return data

def fetch_dbpedia_data(institution_name: str) -> dict:

    dbpedia_url = f"https://dbpedia.org/data/{institution_name.replace(' ', '_')}.json"

    headers = {"User-Agent": "MyInstitutionInfoBot/1.0"}
    response = requests.get(dbpedia_url, headers=headers)

    if response.status_code != 200:

        return {}

    dbpedia_json = response.json()

    entity_url = f"http://dbpedia.org/resource/{institution_name.replace(' ', '_')}"
    data = {"founder": "Unknown"}

    if entity_url in dbpedia_json:

        entity = dbpedia_json[entity_url]

        if "http://dbpedia.org/ontology/foundedBy" in entity:

            data["founder"]=entity["http://dbpedia.org/ontology/foundedBy"]
            [0]["value"]

    return data

def create_institution_profile(institution_name: str) -> InstitutionProfile:

    data = fetch_wikipedia_data(institution_name)

    profile = InstitutionProfile(

        founder=data["founder"],

        Established=data["Established"],

        branches=data["branches"],

        employee_count=data["employee_count"],

        summary=data["summary"], )

    return profile

```

```
if __name__ == "__main__":
    institution_name = input("Enter institution name: ")
    try:
        profile = create_institution_profile(institution_name)
        print(profile.model_dump_json(indent=2))
    except ValueError as e:
        print(e)
```

OUTPUT

Enter institution name: vtu

```
{
  "founder": "Unknown",
  "Established": 1998,
  "branches": [
    "Belagavi"
  ],
  "employee_count": 0,
  "summary": "Visvesvaraya Technological University (VTU), is a collegiate public state university in Belagavi, Karnataka established by the Government of Karnataka. It is one of the largest Technological Universities in India with 26 years of Tradition of excellence in Engineering & Technical Education, Research"
}
```

PROGRAM 10

Build a chat bot for the Indian Penal Code. We'll start by downloading the official Indian Penal Code document, and then we'll create a chat bot that can interact with it. Users will be able to ask questions about the Indian Penal Code and have a conversation with it.

PROGRAM CODE

```
!pip install langchain cohere wikipedia-api pydantic
```

```
!pip install langchain_community
```

```
# Import necessary libraries
```

```
from langchain import PromptTemplate, LLMChain
```

```
from langchain.llms import Cohere
```

```
from pydantic import BaseModel
```

```
from typing import Optional
```

```
import wikipediaapi
```

```
# Step 1: Set up the Cohere API
```

```
import getpass
```

```
COHERE_API_KEY = getpass.getpass('Enter your Cohere API Key: ')
```

```
cohere_llm = Cohere(cohere_api_key=COHERE_API_KEY, model="command")
```

```
# Step 2: Download Indian Penal Code (IPC) summary from Wikipedia
```

```
def fetch_ipc_summary():
```

```
    user_agent = "IPCChatbot/1.0 (contact: myemail@example.com)"
```

```
    wiki_wiki = wikipediaapi.Wikipedia(user_agent=user_agent, language='en')
```

```
    page = wiki_wiki.page("Indian Penal Code")
```

```
    if not page.exists():
```

```
        raise ValueError("The Indian Penal Code page does not exist on  
        Wikipedia.")
```

```
        return page.text[:5000] # Limiting to first 5000 characters for brevity
```

```
ipc_content = fetch_ipc_summary()
```

```
# Step 3: Define a Pydantic model for structured responses
```

```
class IPCResponse(BaseModel):
```

```
    section: Optional[str]
```

```
    explanation: Optional[str]
```


Step 4: Create a prompt template for the chatbot

```
prompt_template = PromptTemplate(
    input_variables=["question"],
    template="""
        You are a legal assistant chatbot specialized in the Indian Penal Code
        (IPC).Refer to the following IPC document content to answer the user's
        query:
        {ipc_content}
        User Question: {question}
        Provide a detailed answer, mentioning the relevant section if
        applicable.
    """)
```

Step 5: Function to interact with the chatbot

```
def get_ipc_response(question: str) -> IPCResponse:
    formatted_prompt = prompt_template.format(ipc_content=ipc_content,
        question=question)
    response = cohere_llm.predict(formatted_prompt)
```

Extracting and structuring the response

```
    if "Section" in response:
        section = response.split('Section')[1].split(':')[0].strip()
        explanation = response.split(':', 1)[-1].strip()
    else:
        section = None
        explanation = response.strip()
    return IPCResponse(section=section, explanation=explanation)
```

Step 6: Set up interactive chatbot in Jupyter

```
from IPython.display import display
```

```

import ipywidgets as widgets

# Function to display chatbot responses

def display_response(response: IPCResponse):
    print(f"Section: {response.section if response.section else 'N/A'}")
    print(f"Explanation: {response.explanation}")

# Function to handle user input

def on_button_click(b):
    user_question = text_box.value
    try:
        response = get_ipc_response(user_question)
        display_response(response)
    except Exception as e:
        print(f"Error: {e}")

# Create text box and button for user input

text_box = widgets.Text(value="",placeholder='Ask about the Indian Penal Code',description='You:',disabled=False)

button=widgets.Button(description='Ask',disabled=False,button_style="",tooltip='Click to ask a question about IPC',icon='legal')

button.on_click(on_button_click)

# Display the chatbot interface

display(text_box, button)

```

OUTPUT

Enter your Cohere API Key:

